



# Top 15 SQL Interview Questions

## The Ultimate Revision Handbook

### Includes:

- ✓ Problem Statements
- ✓ Input Tables
- ✓ SQL Queries
- ✓ Expected Outputs
- ✓ Easy Explanations

**Prepared by Shilpi Yadav**

Data Engineer | SQL • AWS • PySpark • Data Engineering

LinkedIn: [linkedin.com/in/shilpiyadav01](https://www.linkedin.com/in/shilpiyadav01)

## 🕒 SQL Interview Question 1: Find the Highest Salary

### 🔗 Problem:

Find the highest salary from the Employee table.

### 📊 Input Table:

| Emp_ID | Name  | Salary |
|--------|-------|--------|
| 1      | Rahul | 50000  |
| 2      | Aman  | 70000  |
| 3      | Priya | 90000  |
| 4      | Neha  | 65000  |

### 💻 Query:

```
SELECT MAX(Salary) AS Highest_Salary
FROM Employee;
```

### ✅ Output:

| Highest_Salary |
|----------------|
| 90000          |

### 💡 Explanation:

MAX() returns the largest value from a numeric column.

🕒 **Interview Tip:** *This is often used as a warm-up question before asking for the Second Highest Salary.*

---

## 🕒 SQL Interview Question 2: Find the Second Highest Salary

### 🔗 Problem:

Find the second highest salary from the Employee table.

### 📊 Input Table:

| Emp_ID | Name  | Salary |
|--------|-------|--------|
| 1      | Rahul | 50000  |
| 2      | Aman  | 70000  |
| 3      | Priya | 90000  |
| 4      | Neha  | 65000  |

#### Query:

```
SELECT MAX(Salary) AS Second_Highest_Salary
FROM Employee
WHERE Salary <
(
    SELECT MAX(Salary)
    FROM Employee
);
```

#### ✓ Output:

| Second_Highest_Salary |
|-----------------------|
| 70000                 |

#### 💡 Explanation:

First, find the highest salary using a subquery. Then, filter the main query to exclude that maximum value and find the MAX() again.

🔗 **Interview Tip:** Interviewers may ask how to handle duplicate salaries or how to scale this to the Nth highest salary using window functions.

## 🔗 SQL Interview Question 3: Employees Earning Above Average

#### 🔗 Problem:

Find employees whose salary is greater than the average salary.

#### 📄 Input Table:

| Emp_ID | Name  | Salary |
|--------|-------|--------|
| 1      | Rahul | 50000  |
| 2      | Aman  | 70000  |
| 3      | Priya | 90000  |
| 4      | Neha  | 65000  |

Prepared by Shilpi Yadav

Data Engineer | SQL • AWS • PySpark • Data Engineering  
 LinkedIn: [linkedin.com/in/shilpiyadav01](https://www.linkedin.com/in/shilpiyadav01)

### Query:

```
SELECT *  
FROM Employee  
WHERE Salary >  
(  
    SELECT AVG(Salary)  
    FROM Employee  
);
```

### Output:

| Emp_ID | Name  | Salary |
|--------|-------|--------|
| 2      | Aman  | 70000  |
| 3      | Priya | 90000  |

### Explanation:

AVG() calculates the average salary. The outer query filters and returns only the employees whose salary exceeds that calculated average.

**🕒 Interview Tip:** Commonly used to test your understanding of subqueries and aggregate functions.

---

## 🕒 SQL Interview Question 4: Count Employees Department Wise

### Problem:

Count the number of employees in each department.

### Input Table:

| Emp_ID | Name  | Department |
|--------|-------|------------|
| 1      | Rahul | IT         |
| 2      | Aman  | HR         |
| 3      | Priya | IT         |
| 4      | Neha  | HR         |
| 5      | Ravi  | IT         |

### Query:

```
SELECT Department ,  
COUNT(*) AS Total_Employees
```

**Prepared by Shilpi Yadav**

Data Engineer | SQL • AWS • PySpark • Data Engineering  
LinkedIn: [linkedin.com/in/shilpiyadav01](https://www.linkedin.com/in/shilpiyadav01)

```
FROM Employee
GROUP BY Department;
```

#### ✔ Output:

| Department | Total_Employees |
|------------|-----------------|
| IT         | 3               |
| HR         | 2               |

#### 💡 Explanation:

GROUP BY groups the records by department. COUNT(\*) then calculates the number of rows within each specific group.

🕒 **Interview Tip:** *Frequently asked with HAVING clause follow-ups, e.g., "Show departments with more than 2 employees."*

---

## 🕒 SQL Interview Question 5: Find Duplicate Records

#### 🔍 Problem:

Find duplicate email addresses in a table.

#### 📄 Input Table:

| ID | Email       |
|----|-------------|
| 1  | a@gmail.com |
| 2  | b@gmail.com |
| 3  | a@gmail.com |
| 4  | c@gmail.com |

#### 💻 Query:

```
SELECT Email,
COUNT(*) AS Count
FROM Users
GROUP BY Email
HAVING COUNT(*) > 1;
```

#### ✔ Output:

| Email       | Count |
|-------------|-------|
| a@gmail.com | 2     |

#### 💡 Explanation:

HAVING is used to filter grouped records (since WHERE cannot be used with aggregations). It ensures only emails appearing more than once are returned.

🎯 **Interview Tip:** *This is one of the most frequently asked SQL data cleaning questions.*

## 🎯 SQL Interview Question 6: Find Top 3 Salaries

#### 🔗 Problem:

Find the top 3 highest salaries from the company.

#### 📊 Input Table:

| Emp_ID | Name  | Salary |
|--------|-------|--------|
| 1      | Rahul | 50000  |
| 2      | Aman  | 70000  |
| 3      | Priya | 90000  |
| 4      | Neha  | 65000  |

#### 💻 Query:

```
SELECT DISTINCT Salary
FROM Employee
ORDER BY Salary DESC
LIMIT 3;
```

#### ✅ Output:

| Salary |
|--------|
| 90000  |
| 70000  |
| 65000  |

#### 💡 Explanation:

ORDER BY sorts the salaries in descending order (highest first). LIMIT restricts the output

**Prepared by Shilpi Yadav**

Data Engineer | SQL • AWS • PySpark • Data Engineering  
 LinkedIn: [linkedin.com/in/shilpiyadav01](https://www.linkedin.com/in/shilpiyadav01)

to the first 3 records. DISTINCT ensures duplicate salaries don't consume top spots.

🎯 **Interview Tip:** Interviewers may ask for a solution without LIMIT (e.g., using Oracle's ROWNUM, SQL Server's TOP, or window functions).

---

## 🎯 SQL Interview Question 7: Find Employees Without Managers

### 🔗 Problem:

Identify employees who do not have a manager assigned.

### 📊 Input Table:

| Emp_ID | Name  | Manager_ID |
|--------|-------|------------|
| 1      | Rahul | NULL       |
| 2      | Aman  | 1          |
| 3      | Priya | 1          |

### 💻 Query:

```
SELECT Emp_ID, Name
FROM Employee
WHERE Manager_ID IS NULL;
```

### ✅ Output:

| Emp_ID | Name  |
|--------|-------|
| 1      | Rahul |

### 💡 Explanation:

NULL represents an unknown or missing value. Using 'IS NULL' is the correct way to filter for these blank values rather than using '='.

🎯 **Interview Tip:** Always remember that NULL = NULL evaluates to Unknown, which is why IS NULL must be used.

---

## 🎯 SQL Interview Question 8: Customers Who Never Ordered

Prepared by Shilpi Yadav

Data Engineer | SQL • AWS • PySpark • Data Engineering  
LinkedIn: [linkedin.com/in/shilpiyadav01](https://www.linkedin.com/in/shilpiyadav01)

### 🔗 Problem:

Find the customers who have never placed an order.

### 🗃️ Input Table:

#### Customers Table:

| CustomerID | CustomerName |
|------------|--------------|
| 1          | John         |
| 2          | Jane         |
| 3          | Bob          |

#### Orders Table:

| OrderID | CustomerID |
|---------|------------|
| 101     | 1          |
| 102     | 1          |

### 📝 Query:

```
SELECT c.CustomerID ,  
       c.CustomerName  
FROM Customers c  
LEFT JOIN Orders o  
ON c.CustomerID = o.CustomerID  
WHERE o.CustomerID IS NULL;
```

### ✅ Output:

| CustomerID | CustomerName |
|------------|--------------|
| 2          | Jane         |
| 3          | Bob          |

### 💡 Explanation:

LEFT JOIN keeps all customers regardless of matches. The WHERE clause filters out customers who do have matching records in the Orders table, leaving those with NULLs.

🔗 **Interview Tip:** Tests JOIN concepts. You might also be asked to solve this using NOT IN or NOT EXISTS.

## 🔗 SQL Interview Question 9: Department Wise Highest Salary

Prepared by Shilpi Yadav

Data Engineer | SQL • AWS • PySpark • Data Engineering  
LinkedIn: [linkedin.com/in/shilpiyadav01](https://www.linkedin.com/in/shilpiyadav01)

### 🔗 Problem:

Find the highest salary in each department.

### 📊 Input Table:

| Emp_ID | Department | Salary |
|--------|------------|--------|
| 1      | IT         | 50000  |
| 2      | HR         | 70000  |
| 3      | IT         | 90000  |
| 4      | HR         | 65000  |

### 💻 Query:

```
SELECT Department ,  
MAX(Salary) AS Highest_Salary  
FROM Employee  
GROUP BY Department;
```

### ✅ Output:

| Department | Highest_Salary |
|------------|----------------|
| IT         | 90000          |
| HR         | 70000          |

### 💡 Explanation:

Groups the employee records by the unique Department name, then applies the MAX() aggregate function to the salaries inside each specific bucket.

🔗 **Interview Tip:** Remember, every column in the SELECT clause must either be in the GROUP BY clause or be wrapped in an aggregate function.

---

## 🔗 SQL Interview Question 10: Find Nth Highest Salary

### 🔗 Problem:

Find the Nth highest salary from an Employee table (e.g., 3rd highest).

### 📊 Input Table:

| Name  | Salary |
|-------|--------|
| Rahul | 50000  |
| Aman  | 70000  |
| Priya | 90000  |
| Neha  | 65000  |

#### Query:

```
SELECT Salary
FROM
(
    SELECT Salary,
    DENSE_RANK() OVER (ORDER BY Salary DESC) AS rnk
    FROM Employee
) x
WHERE rnk = 3;
```

#### ✓ Output:

| Salary |
|--------|
| 65000  |

#### 💡 Explanation:

DENSE\_RANK() ranks the salaries in descending order. Wrapping it in a subquery (or CTE) allows us to filter where the rank equals exactly 3 (or any 'N').

🔗 **Interview Tip:** *DENSE\_RANK is preferred over RANK here to handle duplicate salaries without skipping any rank numbers.*

## 🔗 SQL Interview Question 11: ROW\_NUMBER() vs RANK() vs DENSE\_RANK()

#### 🔗 Problem:

Show the difference between ROW\_NUMBER(), RANK(), and DENSE\_RANK().

#### 📄 Input Table:

| Salary |
|--------|
| 90000  |
| 90000  |
| 70000  |

**Prepared by Shilpi Yadav**

Data Engineer | SQL • AWS • PySpark • Data Engineering  
 LinkedIn: [linkedin.com/in/shilpiyadav01](https://www.linkedin.com/in/shilpiyadav01)

### Query:

```
SELECT Salary,
ROW_NUMBER() OVER(ORDER BY Salary DESC) AS RN,
RANK() OVER(ORDER BY Salary DESC) AS Rnk,
DENSE_RANK() OVER(ORDER BY Salary DESC) AS D_Rnk
FROM Employee;
```

### Output:

| Salary | ROW_NUMBER | RANK | DENSE_RANK |
|--------|------------|------|------------|
| 90000  | 1          | 1    | 1          |
| 90000  | 2          | 1    | 1          |
| 70000  | 3          | 3    | 2          |

### Explanation:

**ROW\_NUMBER** → Assigns a unique sequential number.

**RANK** → Assigns same rank to ties, but *skips* the next number.

**DENSE\_RANK** → Assigns same rank to ties, with *no gaps*.

🎯 **Interview Tip:** *This is an absolute must-know window function concept. Memorize this behavior!*

---

## 🎯 SQL Interview Question 12: Find Latest Order Per Customer

### Problem:

Find the most recent order date for every customer.

### Input Table:

| Order_ID | Customer_ID | Order_Date |
|----------|-------------|------------|
| 101      | 1           | 2023-01-01 |
| 102      | 1           | 2023-02-15 |
| 103      | 2           | 2023-03-10 |

### Query:

```
SELECT Customer_ID,
MAX(Order_Date) AS Latest_Order
FROM Orders
```

**Prepared by Shilpi Yadav**

Data Engineer | SQL • AWS • PySpark • Data Engineering  
LinkedIn: [linkedin.com/in/shilpiyadav01](https://www.linkedin.com/in/shilpiyadav01)

```
GROUP BY Customer_ID;
```

#### ✔ Output:

| Customer_ID | Latest_Order |
|-------------|--------------|
| 1           | 2023-02-15   |
| 2           | 2023-03-10   |

#### 💡 Explanation:

By grouping by the Customer ID and finding the Maximum (latest) date, we extract the most recent transaction time.

🕒 **Interview Tip:** *If you need the entire order row (e.g., the amount of that latest order), you will need to use a window function or a self-join.*

---

## 🕒 SQL Interview Question 13: Calculate a Running Total

#### 🔗 Problem:

Calculate the cumulative sum (running total) of order amounts.

#### 📊 Input Table:

| Order_ID | Amount |
|----------|--------|
| 1        | 100    |
| 2        | 150    |
| 3        | 200    |

#### 💻 Query:

```
SELECT Order_ID,
       Amount,
       SUM(Amount) OVER (ORDER BY Order_ID) AS Running_Total
FROM Orders;
```

#### ✔ Output:

| Order_ID | Amount | Running_Total |
|----------|--------|---------------|
| 1        | 100    | 100           |
| 2        | 150    | 250           |
| 3        | 200    | 450           |

#### 💡 Explanation:

The SUM() window function calculates the total amount, and the ORDER BY inside the OVER() clause defines the expanding window frame to create the cumulative effect.

🔗 **Interview Tip:** Adding PARTITION BY inside the OVER() clause allows you to reset the running total per group (e.g., per month or per customer).

## 🔗 SQL Interview Question 14: Find Missing IDs

#### 🔗 Problem:

Find the missing sequential ID in a table.

#### 📄 Input Table:

| ID |
|----|
| 1  |
| 2  |
| 4  |
| 5  |

#### 💻 Query:

```
SELECT a.ID + 1 AS Missing_ID
FROM Sequence_Table a
LEFT JOIN Sequence_Table b
  ON a.ID + 1 = b.ID
WHERE b.ID IS NULL
      AND a.ID < (SELECT MAX(ID) FROM Sequence_Table);
```

#### ✅ Output:

| Missing_ID |
|------------|
| 3          |

#### 💡 Explanation:

This checks the next expected ID (current ID + 1). If a LEFT JOIN on that expected ID returns NULL, it means the sequence is broken and that ID is missing.

🎯 **Interview Tip:** A very common data-quality problem. Another modern approach is to use the LEAD() window function.

## 🎯 SQL Interview Question 15: Top 3 Employees Per Department

### 🔗 Problem:

Find the top 3 highest earning employees within each department.

### 📊 Input Table:

| Emp_ID | Name  | Department | Salary |
|--------|-------|------------|--------|
| 1      | Rahul | IT         | 50000  |
| 2      | Aman  | IT         | 90000  |
| 3      | Priya | HR         | 70000  |
| 4      | Neha  | HR         | 85000  |
| 5      | Ravi  | IT         | 80000  |
| 6      | Yash  | IT         | 40000  |

### 💻 Query:

```
SELECT *
FROM
(
    SELECT *,
    DENSE_RANK() OVER (
        PARTITION BY Department
        ORDER BY Salary DESC
    ) AS rnk
    FROM Employee
) x
WHERE rnk <= 3;
```

### ✅ Output:

| Emp_ID | Name  | Department | Salary | rnk |
|--------|-------|------------|--------|-----|
| 2      | Aman  | IT         | 90000  | 1   |
| 5      | Ravi  | IT         | 80000  | 2   |
| 1      | Rahul | IT         | 50000  | 3   |
| 4      | Neha  | HR         | 85000  | 1   |
| 3      | Priya | HR         | 70000  | 2   |

Prepared by Shilpi Yadav

Data Engineer | SQL • AWS • PySpark • Data Engineering  
LinkedIn: [linkedin.com/in/shilpiyadav01](https://www.linkedin.com/in/shilpiyadav01)

### 💡 **Explanation:**

PARTITION BY creates separate ranking groups for each department. ORDER BY sorts the salaries within those partitions. We then filter for ranks 1, 2, and 3.

🎯 **Interview Tip:** *An absolute classic Data Engineer and SQL Developer interview question. Master PARTITION BY!*

# SQL Interview Quick Revision

---

| Topic                                   | Must Know |
|-----------------------------------------|-----------|
| Joins (INNER, LEFT, SELF)               | ✓         |
| CTE (Common Table Expressions)          | ✓         |
| Window Functions (RANK, LEAD, LAG)      | ✓         |
| Subqueries (Correlated vs Uncorrelated) | ✓         |
| Aggregations (GROUP BY, HAVING)         | ✓         |

**Prepared by Shilpi Yadav**

Data Engineer | SQL • AWS • PySpark • Data Engineering  
LinkedIn: [linkedin.com/in/shilpiyadav01](https://www.linkedin.com/in/shilpiyadav01)

# Thank You!

Good luck with your SQL interviews! You've got this.

**Follow for more resources on:**

 SQL  AWS  PySpark  Data Engineering

---

**Shilpi Yadav**

*Data Engineer*

**LinkedIn:** [linkedin.com/in/shilpiyadav01](https://www.linkedin.com/in/shilpiyadav01)

**Prepared by Shilpi Yadav**

Data Engineer | SQL • AWS • PySpark • Data Engineering  
LinkedIn: [linkedin.com/in/shilpiyadav01](https://www.linkedin.com/in/shilpiyadav01)